Component 4 (Traceability Agent): status summary

Status as of 2026-10-09. Everything below was measured or run in this repository unless marked otherwise.

1. What was built

Pipeline: load artifacts from Neo4j (USKG) -> clean and chunk text -> hybrid retrieval (TF-IDF + embeddings,

top-k) -> LLM verification (Llama 3.1 8B, "software traceability expert" prompt) -> write `VERIFIED_TRACE`

edges -> monitor decay (`DecayFlag` + Redis event) -> completeness score and orphan detection.

Area	What exists
Schema alignment	`Requirement` / `CodeEntity` / `VERIFIED_TRACE` / `DecayFlag` per the system spec; Neo4j indexes (`setup` command)
Retrieval (recall)	Code cleaning, identifier splitting, 60-word chunking, TF-IDF + MiniLM fused by z-score
Verification (precision)	JSON-mode LLM client, expert and generic prompts, confidence threshold, per-call latency recorded
Graph writes	Idempotent `MERGE`, vetted write statements only (ownership guard), links written as they are confirmed
Decay	Hash-based change detection, re-verify, mark decayed without deleting, `DecayFlag`, Redis publisher
Completeness	Score 0-100, orphan list, `score` command and dashboard panel
Command line	`python -m agents.traceability_agent setup / link / monitor / score`, plus `evaluate` and `decay_injection`
Dashboard	FastAPI + web page: live run feed, precision/recall, trace graph, Neo4j status, run start/stop, completeness
Self-hosted LLM	OpenAI-compatible client config; Ollama on a CPU server behind an nginx API-key proxy
Datasets	iTrust (requirement to code, primary) and CM-1 converted to our schema, with full gold labels
Tests	92 tests: 85 pass, 7 real-Neo4j integration tests skip while Neo4j is down
Docs	Agent README, report draft (`docs/traceability_report_draft.md`)

2. Completion

My own estimate: **about 80% of the build, about 45% of the evaluation.**

Functional requirements (9 in the spec): **8 implemented and tested, 1 untested live.**

#	Requirement	Status
1	Candidate links by embedding similarity	Done
2	LLM verification with confidence 0-1	Done
3	Plain-language justification	Done
4	Store `VERIFIED_TRACE` edges	Done (real-Neo4j tested)
5	Re-evaluate on change	Done (real-Neo4j tested with a stubbed LLM)
6	Mark decayed, keep the edge	Done (real-Neo4j tested)

#	Requirement	Status
7	Real-time decay notification	Implemented; **never run against a real Redis server**
8	Completeness score	Done
9	Orphan requirements	Done

Non-functional targets: **1 of 4 met.**

Target	Result
Scale (10,000 nodes, queries < 2 s)	**Met**: 8 ms write, 4 ms read of traces, 0.48 s to read all code entities
LLM latency <= 5 s	**Not met**: typical 52 s on the CPU server (needs GPU or hosted LLM)
Precision >= 90%	**Not met**: 0.375 on the iTrust sample
Recall >= 80%	**Not met**: stage 1 reaches 41.9% of true links at top-10 (68.4% at top-50)

3. Measured results

Stage-1 recall ceiling on iTrust (399 true links): 9.0% at first (MiniLM, raw code) -> 41.9% at top-10 after cleaning, hybrid fusion and chunking.

Full pipeline vs baselines (iTrust, 10 requirements, top-5, 35 true links):

Method	Precision	Recall	F1
Embeddings only	0.060	0.086	0.071
TF-IDF only	0.120	0.171	0.141
Hybrid retrieval, no LLM	0.140	0.200	0.165
Full pipeline	**0.375**	0.171	**0.235**

The LLM stage raised precision 2.7x but cannot exceed the shortlist's recall. One small sample: treat as a first signal.

4. What is not done

Item	Why
Persona ablation (expert vs generic prompt)	The batch run crashed at 14 of 50 calls on an nginx `502` from the LLM server. Partial data (6 links kept) is not comparable.
Real graph write run (CM-1)	Crashed at 29 of 40 on a connection error, **before any link was written** (now fixed: links are saved immediately and a server failure no longer loses work)
Decay-injection test (target >= 85%)	Had no verified links to test, because the step above failed
Redis end to end	`redis` package not installed, no Redis server
Neo4j currently offline	The server is not running on this machine, so the real-graph tests are skipped
Larger evaluation (100-150 pairs)	Needs a faster LLM; 10 requirements is below the plan
Team confirmations	C1/C2 property names (assumed `text`, `kind`), C3 Evidence/RVU structure, Redis message format

Item	Why
Report	Draft exists; sections 5.4-5.6 await the runs above; architecture diagram is in Mermaid and still needs exporting
Git	Work is committed on `feat/traceability-agent-setup` (not `feature/traceability-agent`); the latest fixes are uncommitted

5. Next steps, in order

1. Start Neo4j; run `python -m agents.traceability_agent setup`, then the real-Neo4j tests (7 should run).
2. Rerun `bash scripts/final_evaluation.sh` (about 2 hours): persona ablation, CM-1 link writes, decay injection.
3. Install Redis (`pip install redis`, a local Redis server) and run one real decay notification.
4. Confirm schema names and the Redis event format with C1, C2 and C3.
5. If time allows: a stronger LLM (GPU or hosted) to test the precision and latency targets, and a larger sample.
6. Fill report sections 5.4-5.6, export the diagram, commit and open the pull request.